# Dominican Express — Funcionalidades y Justificación de SurrealDB

2026-09-21 · @Someone

## Contexto

Dominican Express es el proyecto de la asignatura de Bases de Datos Avanzadas (UFV): una aplicación bancaria web y móvil inspirada en el modelo de American Express. Este documento recoge las funcionalidades básicas definidas para la app y la justificación de por qué SurrealDB (una base de datos NoSQL multi-modelo) encaja con ellas, para compartir con el grupo antes de diseñar el esquema.

## Funcionalidades básicas

### Cuentas y tarjetas

- Cada usuario tiene una o varias cuentas bancarias, y cada cuenta puede tener varias tarjetas asociadas.
- Consulta de datos de cuenta: número, saldo, divisa, nombre del titular.
- Consulta de datos de tarjeta: número, caducidad y CVV (bajo verificación).
- Historial de transacciones por cuenta y por tarjeta.

### Movimientos

- Transferencias entre cuentas del propio banco.
- Envío de bizums a otros usuarios del banco.

### Cuentas compartidas

- Cuentas con varios cotitulares.

### Notificaciones (app móvil)

- Aviso al recibir dinero, al ingresar la nómina, o al producirse un gasto.

### Funciones adicionales propuestas

- Bloqueo/desbloqueo de tarjeta.
- Límites de gasto o retirada configurables.
- Favoritos/contactos frecuentes para transferencias y bizum.

- Domiciliaciones o pagos recurrentes.
- Categorización de gastos.
- Filtro/búsqueda en el historial y exportación de extractos.
- Alertas de saldo bajo y de movimientos sospechosos.
- Opcionales: objetivos de ahorro, conversión de divisa, simulador de préstamo/crédito.

## Por qué SurrealDB

SurrealDB es una base de datos NoSQL multi-modelo (documento + grafo) que, a diferencia de otros NoSQL, ofrece varias características poco habituales y muy relevantes para un caso bancario:

- **Multi-modelo (documento + grafo):** permite modelar entidades como documentos (cuentas, tarjetas) y relaciones complejas como aristas de grafo (cotitularidad, transferencias), sin forzar todo a tablas planas ni a un único modelo.
- **Transacciones ACID:** aunque es NoSQL, garantiza consistencia fuerte, algo imprescindible en un banco (una transferencia no puede quedar a medias).
- **Live queries (tiempo real):** permite suscribirse a cambios en los datos y recibir eventos en tiempo real, ideal para las notificaciones push sin infraestructura adicional.
- **Esquema flexible por tabla:** se puede usar SCHEMAFULL en tablas críticas (cuentas, transacciones) y SCHEMALESS en las menos críticas, sin renunciar a integridad donde importa.
- **Permisos a nivel de campo:** útil para proteger datos sensibles como el CVV o el número de tarjeta, mostrándolos solo bajo ciertas condiciones.
- **SurrealQL cercano a SQL:** reduce la curva de aprendizaje del equipo frente a otros NoSQL como MongoDB o Cassandra.
- **Escalabilidad distribuida:** pensada para crecer en volumen de transacciones.

## Mapeo funcionalidad → característica de SurrealDB

| Funcionalidad | Necesidad de modelado | Característica de SurrealDB |
|---|---|---|
| Cuentas compartidas con cotitulares | Relación muchos-a-muchos usuario–cuenta | Grafo ( `RELATE` ) con atributos en la arista (rol: titular/cotitular) |

| Funcionalidad | Necesidad de modelado | Característica de SurrealDB |
|---|---|---|
| Transferencias y bizum | Relación con atributos entre cuentas/usuarios, posible análisis de patrones | Grafo ( `RELATE` ) + travesias para detectar patrones de fraude |
| Notificaciones push en tiempo real | Reacción inmediata a cambios en los datos | Live Queries |
| Datos sensibles de tarjeta (número, CVV) | Control de acceso a campos concretos | Permisos a nivel de campo + tablas SCHEMAFULL |
| Historial de transacciones y datos de cuenta | Consulta estructurada de campos fijos | Modelo de documento + travesia de aristas de transferencia |
| Favoritos/contactos frecuentes | Relación usuario–usuario o usuario–cuenta reutilizable | Grafo ( `RELATE` ), igual que la cotitularidad |

## Conclusión y próximos pasos

Con esta base, el siguiente paso es diseñar el esquema concreto en SurrealQL (tablas, relaciones y permisos) y repartir las tareas de implementación del backend en Java. Quedan dos decisiones abiertas para hablar en grupo: el alcance final de las funciones adicionales (¿cuáles entran en la primera entrega?) y el nivel de detalle del análisis de patrones de transacciones/fraude.